

document	title	author	date	target	audience
P2616R4	Making std::atomic notification/wait operations usable in more situations	Lewis Baker lewissbaker@gmail.com	2023- 02-15	C++26	LEWG

- [Abstract](#)
- [Motivation](#)
- [Proposed design](#)
- [Design discussion](#)
- [Potential Implementation Strategies](#)
- [Proposed Wording](#)
- [References](#)

Revision History

R4

- Apply feedback from SG1 review in Issaquah 2023.
- Remove deprecation of existing atomic notify functions as SG1 feedback indicated there may still be some use-cases for them.
- Move mention of lifetime in the effects of `notify_token::notify_one/all` into a non-normative note.
- Fix non-escaped angle brackets in wording synopsis.
- Add wording to be applied to P2689R1 if both are accepted.

R3

- Rename `atomic_notify_token` to `notify_token` and change template parameter to be the atomic-object type rather than the value-type.

R2

- Reduce paper down to option 3 only at request of SG1
- Rephrase abstract
- Add design discussion
- Add proposed wording
- Extend changes to `atomic_flag`

R1

- Added Option 3 - `atomic_notify_token` approach.
- Incorporate feedback on limitations of option 2.

R0

Proposed two options.

- Option 1 - allow `std::atomic_notify_one/all()` on potentially destroyed pointer
- Option 2 - add fused modify + notify overloads, `std::memory_notification` enum

Abstract

C++20 added support to `std::atomic` for `notify_one()`, `notify_all()` and `wait()` operations which give applications an efficient, light-weight way to block until the value of an atomic object reaches a certain value.

A waiting thread calls the `x.wait(oldValue)` to wait until the atomic object, `x`'s value changes from `oldValue` to something else and a signalling thread first stores a new value to the atomic object, `x`, and then calls one of the notify-methods to wake up any threads that have blocked inside a call to `wait()`.

For use-cases where a waiting thread can go on to destroy the atomic object once it has observed the store, the signalling thread's subsequent call to the atomic object's `notify_one()` or `notify_all()` member function can potentially have undefined behaviour. This is because the standard [basic.life] p6.2 states that a call to a member function on a pointer to an object whose lifetime has ended has undefined behaviour.

This paper proposes introducing a new API for obtaining a `std::notify_token<Atomic>` from a `std::atomic<T>`, `std::atomic_ref<T>` or `std::atomic_flag` which can then be

used to notify threads waiting on that atomic object without worrying about whether the underlying atomic object is still alive.

This paper does not propose deprecating or changing the specification of

`std::atomic_[flag_]notify_one/all()` at this time.

Usages of the namespace-scope functions will still have potential for undefined behaviour in some cases, however. Resolving these issues is deferred pending the outcome of core language changes proposed by the "pointer zap" papers (P1726 and P2188) with regards to pointer provenance.

This paper does not attempt to address the undefined behaviour C compatibility layer for

`std::atomic`.

Motivation

Consider the following example:

```
int main() {
    thread_pool tp;

    {
        std::atomic<int> x{0}; // #1
        tp.execute([&] {      // #2
            x.store(1);        // #3
            x.notify_one();    // #4
        });
        x.wait(0);            // #5
        assert(x.load() == 1);
    }                          // #6
}
```

Let T1 be the thread executing `main()` and T2 be the thread-pool thread that executes the lambda.

We might end up with a situation where the following sequence of events is executed:

```
T1: constructs the atomic `x` and initialises to 0 (#1)
T1: enqueues the task to execute on the T1 (#2)
T2: dequeues the task and invokes the lambda
T2: stores the value 1 to the `x` (#3)
T1: executes `x.wait(0)`, sees value is now 1 and returns immediately (#5)
```

T1: destroys the object `x` (#6)
T2: executes `x.notify\_one()` on a now destroyed object `x` (#4)

The final call to `x.notify_one()` member function on a destroyed object in (#4) has undefined behaviour.

To work around this potential for operating on a dangling reference, we can use two separate atomic variables - `wait()` on one and then spin-wait on the other.

```
int main() {
    thread_pool tp;

    {
        std::atomic<int> x{0};
        std::atomic_flag f{true};
        tp.execute([&] {
            x.store(1);
            x.notify_one();
            f.clear();
        });
        x.wait(0);           // efficient-wait
        while (f.test());    // spin-wait
        assert(x.load() == 1);
    }
}
```

This has a couple of downsides, however:

- It needs to perform two atomic operations instead of one
- The spin-wait can still result in occasional long busy-wait times, wasting CPU resources e.g. if the signalling thread is context-switched out by the OS after the call to `x.notify_one()` but before the call to `f.clear()`, the main thread will busy-wait spin until the signalling thread is rescheduled.

For this particular situation, we could instead use a `std::binary_semaphore`, as its `acquire()` and `release()` operations are defined as being atomic, and thus they do not have the same lifetime issues as the first `std::atomic` implementation.

```
int main() {
    thread_pool tp;

    {
        std::binary_semaphore sem{0};
        tp.execute([&] {
```

```
        sem.release();
    });
    sem.acquire();
}
}
```

However, if we then ask the question "Can we implement `std::counting_semaphore` using `std::atomic`?", we quickly run into the same lifetime questions regarding use of `notify_one/all()` methods.

Indeed, if we look at some of the standard library implementations of `std::counting_semaphore` we see that they do actually follow the same pattern as above for the `release()` operation - an atomic store followed by a call to either `notify_all()` or `notify_one()` on the atomic object.

- See [libc++ `counting\_semaphore::release\(\)` implementation](#).
- See [libstdc++ `counting\_semaphore::release\(\)` implementation](#)
- See [msvc `counting\_semaphore::release\(\)` implementation](#)

So why don't they run into the same lifetime issues?

The reason is that in all major standard library implementations of `std::atomic::notify_all()` and `std::atomic::notify_one()` depend only on the address of the atomic object, but do not actually access any data-members of the atomic object. These platforms also do not seem to target architectures that enforce pointer provenance rules.

The underlying implementations of these notifying functions typically boil down to a call to:

- `WakeByAddressSingle()` / `WakeByAddressAll()` on Windows platforms
- `futex()` with `futex_op` set to `FUTEX_WAKE` on Linux platforms
- `__unlock_wake()` on Mac OS platforms
- Use of one of a collection of statically-allocated condition-variables, with the particular one chosen based on the bit-representation of the provided address.

All of these depend only on the bit-representation of address of the atomic variable, which makes calling `count.notify_all()` work, even though the `count` object may have since been destroyed, as it doesn't try to dereference the pointer.

Standard library implementations have extra knowledge about the semantics of the `std::atomic::notify_one/all()` methods which they can leverage to allow safely implementing `std::counting_semaphore` in terms of `std::atomic` operations. However, users of the standard library which want to be portable cannot make these same

assumptions - the standard library specification does not currently require that all implementations have this same behaviour, and so users must resort to other, more portable means.

Instead of requiring users to work-around these limitations of the interface, we should provide some standard/portable way for users to safely modify an atomic value and notify waiting threads of the change.

Proposed design

This paper proposes the following:

- Adding the `std::notify_token<Atomic>` class template which provides an interface for safely notifying threads waiting on the value to change.
- Adding the `get_notify_token()` member function to `std::atomic<T>`, `std::atomic_ref<T>` and `std::atomic_flag` types for obtaining a notify-token for the corresponding atomic object.

Synopsis:

```
namespace std {
    template<typename Atomic>
    class notify_token;

    template<typename T>e
    class atomic {
    public:
        // Existing members...

        void notify_one() noexcept;
        void notify_one() volatile noexcept;
        void notify_all() noexcept;
        void notify_all() volatile noexcept;

        notify_token<atomic<T>> get_notify_token() noexcept;
        notify_token<volatile atomic<T>> get_notify_token() volatile noexcept;
    };

    template<typename T>
    class atomic_ref {
    public:
        // Existing members...

        void notify_one() noexcept;
        void notify_all() noexcept;
    };
}
```

```

    notify_token<atomic_ref<T>> get_notify_token() noexcept;
};

class atomic_flag {
public:
    // Existing members...

    void notify_one() noexcept;
    void notify_one() volatile noexcept;
    void notify_all() noexcept;
    void notify_all() volatile noexcept;

    notify_token<atomic_flag> get_notify_token() noexcept;
    notify_token<volatile atomic_flag> get_notify_token() volatile noexcept;
};

template<typename Atomic>
class notify_token {
public:
    // Copyable
    notify_token(const notify_token&) noexcept = default;
    notify_token& operator=(const notify_token&) noexcept = default;

    // Perform notifications
    void notify_one() const noexcept;
    void notify_all() const noexcept;
private:
    // exposition-only
    friend Atomic;
    explicit notify_token(std::uintptr_t p) noexcept : address(p) {}
    std::uintptr_t address;
};
}

```

The API would be used as follows:

```

int main() {
    thread_pool tp;

    {
        std::atomic<int> x{0};
        tp.execute([&] {
            // Obtain a notify-token while the object is definitely still alive.
            auto tok = x.get_notify_token();

            // Perform the store - this may cause wait() to return and the main
            // thread to destroy 'x'.

```

```
x.store(1);

// `x' is potentially destroyed from this point on

// Safely notify any objects still waiting on `x' (if any)
tok.notify_one();
});
x.wait(0);
assert(x.load() == 1);
}
}
```

Design discussion

Why does this solution avoid the undefined behaviour?

Current implementations of `std::atomic` notify/wait mechanisms use some kind of hash-table of synchronisation primitives for recording waiting threads and waking them up.

This could either be implemented on the kernel-side of the underlying OS in a `futex()` -like API, or could be implemented in user-space using standard C++ synchronisation primitives.

When obtaining a notify-token, the implementation can perform any necessary hashing of the atomic object address at the time that `get_notify_token()` is called, when the atomic object is still known to be alive.

Then later, when actually performing the notification the implementation can lookup the corresponding hash-table entry to use for synchronisation using the pre-hashed address without needing the address of a potentially destroyed atomic object.

For implementations where the hashing of the address is done in the OS, they would need to at least define it as valid to pass the address of a potentially destroyed object to the syscall.

Conditional notification

This design still retains the ability to conditionally notify waiting threads based on the result of a read-modify-write operation.

This would not be possible were we to take a fused store-and-notify approach.

Why is `std::notify_token` a template?

The strategy used for notify/wait can vary depending on the type of value being synchronised.

For example, on Linux the `futex()` API on which notifications are based only works natively for 32-bit values. When notifying/waiting for types that are not 32-bits in size, the implementation needs to use a proxy 32-bit value which is incremented for every call to notify.

As different types of atomics may need to have different strategies for notification, the `notify_token` needs to be a type that depends on the type of atomic value.

Having the template parameter be the type of the atomic object rather than the value-type of the atomic object allows for different implementation strategies for `atomic<T>` and `atomic_ref<T>` (some `atomic_ref<T>` implementations might use an external lock while `atomic<T>` might use an internal lock).

It also allows having different specialisations for `Atomic` and `volatile Atomic` in case implementations need the flexibility to do something different for notifying volatile-qualified atomic objects.

Const-qualification of `get_notify_token()` and `notify_token` methods

The status quo is that the `std::atomic` methods `notify_one()` and `notify_all()` methods are non-const and thus are only callable on a non-const `std::atomic` value.

This proposal preserves the requirement that the user cannot notify a const-qualified atomic by marking the `get_notify_token()` method as non-const. This prevents the user from obtaining a token if they only have a const-reference to an atomic object.

Once the user has obtained a notify-token, they have shown that they have non-const access to the atomic object.

The notify-token member-functions do not need to also be non-const to enforce const-correctness as the notify-tokens effectively have pointer-semantics. Marking the member-functions as non-const would be easily worked around by just copying the token.

Where should the notify-token type be defined?

This paper proposes introducing a new namespace-scope class template `std::atomic_notify_token<T>`.

This type is needed by methods in both `std::atomic<T>` and `std::atomic_ref<T>`.

An alternative to consider is defining a nested `std::atomic<T>::notify_token` type and then defining `std::atomic_ref<T>::notify_token` as a type-alias of `std::atomic<T>::notify_token`. However, this would have the consequence of forcing an template instantiation of `std::atomic<T>` for every template instantiation of `std::atomic_ref<T>`, even if it's not used.

Naming of `notify_token`

Is the use of the term "token" here consistent with other usages in the standard library? e.g. `std::stop_token`, `std::barrier::arrival_token`

A brief discussion of naming in SG1 Kona 2022 indicated there was no concern about reserving the name `notify_token` for use by the atomic notify/wait mechanisms.

An alternative could be to name this `atomic_notify_token`, although this would then tend to duplicate the term "atomic" within specialisations. e.g.

```
std::atomic_notify_token<std::atomic<int>>
```

Fixing `std::atomic_notify_one/all()`

This proposal does not attempt to change the definition of the namespace-scope functions `std::atomic_notify_one/all()`.

These functions are invoked with a pointer to the atomic object rather than being member functions. So unlike a call to the notify member-functions, which has undefined behaviour if the object's lifetime has ended, the behaviour of passing a pointer to an object whose lifetime has ended is implementation-defined.

So while some implementations might define passing a pointer to an object whose lifetime has ended as valid, other implementations might choose to trap on such uses of a pointer, making writing portably correct code using this API difficult/impossible.

Whether or not this can be made portably safe depends on the resolution to the "pointer zap" issue, which is the subject of papers P1726 and P2188, and so I do not attempt to address the issue in this paper.

If desired, the existing namespace-scope functions could be deprecated and new notify-token-based replacements added. However, it has not yet been explored what the impacts would be on compatibility with C atomics.

In the meantime, we could consider changing the wording of

`std::atomic_notify_one/all()` to no longer be in terms of the corresponding atomic member functions so that we can at least allow the correctness to be implementation-defined instead of undefined-behaviour for the end-of-lifetime cases.

Potential Implementation Strategies

Platforms without native OS support

On platforms without native OS support for address-based notification, the notify/wait mechanisms of `std::atomic` could be implemented in terms of existing synchronisation primitives.

For example: Given the following `__wait_state` helper class definition

```
struct __wait_state {
    std::atomic<uint64_t> _M_waiters{0};
    std::mutex _M_mut;
    std::condition_variable _M_cv;
    std::uint64_t _M_version{0};

    // Get the wait state for a given address.
    static __wait_state& __for_address(volatile void* __address) noexcept {
        constexpr std::uintptr_t __count = 16;
        static __wait_state __w[__count];
        auto __key = (reinterpret_cast<std::uintptr_t>(__address) >> 2) % __count;
        return __w[__key];
    }

    void __notify() noexcept {
        if (_M_waiters.load() != 0) {
            {
                std::lock_guard __lk{_M_mut};
                ++_M_version;
            }
            _M_cv.notify_all();
        }
    }

    template<typename _Pred>
    void __wait(_Pred __pred) noexcept {
        for (int __i = 0; __i < 10; ++__i) {
            if (__pred()) return;
            __yield_thread();
        }
    }
};
```

```

    }

    _M_waiters.fetch_add(1, std::memory_order_seq_cst);
    std::uint64_t __prev_version = [&] {
        std::unique_lock __lk{ _M_mut };
        return _M_version;
    }();
    while (!__pred()) {
        std::unique_lock __lk{ _M_mut };
        if (_M_version == __prev_version) {
            _M_cv.wait(__lk);
        }
        __prev_version = _M_version;
    }
    _M_waiters.fetch_sub(1, std::memory_order_release);
}
};

```

The atomic notify/wait methods can then be defined as follows:

```

namespace std {

template<typename Atomic>
class notify_token {
public:
    void notify_one() noexcept { __state->__notify(); }
    void notify_all() noexcept { __state->__notify(); }
private:
    friend Atomic;
    explicit atomic_notify_token(__wait_state& __state) noexcept
        : __state_(&__state) {}
    __wait_state* __state_;
};

template<typename T>
class atomic {
public:

    void notify_one() noexcept {
        get_notify_token().notify_one();
    }

    void notify_all() noexcept {
        get_notify_token().notify_all();
    }

    notify_token<atomic<T>> get_notify_token() noexcept {
        __wait_state& __s = __wait_state::__for_address(this);
    }
};

```

```

    return atomic_notify_token<T>{__s};
}

void wait(T __old, memory_order __mo = memory_order_seq_cst) const noexcept
    auto __pred = [__mo, __old, this]() noexcept { return this->load(__mo) !=
    auto& __s = __wait_state::__for_address(this);
    __s.wait(__pred);
}

// NOTE: volatile overloads omitted for brevity.
};

} // namespace std

```

In usage of this implementation, the `__wait_state*` object is computed from the address of the atomic object during the call to `get_notify_token()`, while the atomic object is still alive.

Once the address of the `__wait_state` object is computed there is no need for the atomic object address any more - and thus no need to use a pointer to a potentially destroyed object.

Platforms with native OS support

On platforms that use a `futex()`-like OS API to implement waiting, the token could hold the address of the object (e.g. as a `void*` or `std::uintptr_t`) assuming on this particular implementation that it defines it as valid to pass a pointer to a potentially destroyed object.

Example: Possible implementation on Windows

```

namespace std {

template<typename Atomic>
class notify_token {
public:
    void notify_one() noexcept { WakeByAddressSingle(ptr); }
    void notify_all() noexcept { WakeByAddressAll(ptr); }
private:
    friend Atomic;
    explicit notify_token(void* p) noexcept : ptr(p) {}
    void* ptr;
};

template<typename T>
class atomic {

```

```

public:
    void notify_one() noexcept {
        get_notify_token().notify_one();
    }

    void notify_all() noexcept {
        get_notify_token().notify_all();
    }

    atomic_notify_token<T> get_notify_token() noexcept {
        return atomic_notify_token<T>{this};
    }

    void wait(T __old, memory_order __mo = memory_order_seq_cst) noexcept {
        while (load(__mo) == __old) {
            WaitOnAddress(this, std::addressof(__old), sizeof(T), INFINITE);
        }
    }

    // NOTE: volatile overloads omitted for brevity
};

}

```

Proposed Wording

Modify `[atomics.wait]` Note 3 as follows:

[Note 3: The following functions are atomic notifying operations:

- `atomic<T>::notify_one` and `atomic<T>::notify_all`,
- `atomic_flag::notify_one` and `atomic_flag::notify_all`,
- `atomic_notify_one` and `atomic_notify_all`,
- `atomic_flag_notify_one` and `atomic_flag_notify_all`, and
- `atomic_ref<T>::notify_one` and `atomic_ref<T>::notify_all`, and
- `notify_token<Atomic>::notify_one` and `notify_token<Atomic>::notify_all`, and

— end note]

Modify `[atomics.ref.generic.general]` as follows:

```

namespace std {
    template<class T> struct atomic_ref {
    private:
        T* ptr;                // exposition only
    public:
        ...

        void wait(T, memory_order = memory_order::seq_cst) const noexcept;
        void notify_one() const noexcept;
        void notify_all() const noexcept;
        notify_token<atomic_ref<T>> get_notify_token() const noexcept;
    };
}

```

Modify `[atomics.ref.ops]` as follows

Add following at end of the section:

```
notify_token<atomic_ref<T>> get_notify_token() const noexcept;
```

Effects: None.

Returns: An atomic notify token that can be used to unblock the execution of atomic waiting operations on `*ptr`.

Modify `[atomics.ref.int]` as follows:

```

namespace std {
    template<> struct atomic_ref<integral> {
    private:
        integral* ptr;          // exposition only
    public:
        ...

        void wait(integral, memory_order = memory_order::seq_cst) const noexcept;
        void notify_one() const noexcept;
        void notify_all() const noexcept;
        notify_token<atomic_ref<integral>> get_notify_token() const noexcept;
    };
}

```

Modify `[atomics.ref.float]` as follows:

```
namespace std {
    template<> struct atomic_ref<floating-point> {
    private:
        floating-point* ptr;           // exposition only
    public:
        ...

        void wait(floating-point, memory_order = memory_order::seq_cst) const noexcept;
        void notify_one() const noexcept;
        void notify_all() const noexcept;
        notify_token<atomic_ref<floating-point>> get_notify_token() const noexcept;
    };
}
```

Modify `[atomics.ref.pointer]` as follows:

```
namespace std {
    template<class T> struct atomic_ref<T*> {
    private:
        T* ptr;                       // exposition only
    public:
        ...

        void wait(T*, memory_order = memory_order::seq_cst) const noexcept;
        void notify_one() const noexcept;
        void notify_all() const noexcept;
        notify_token<atomic_ref<T*>> get_notify_token() const noexcept;
    };
}
```

Modify `[atomics.types.generic.general]` as follows:

```
namespace std {
    template<class T> struct atomic {
    public:
        ...

        void wait(T*, memory_order = memory_order::seq_cst) const noexcept;
```

```

void notify_one() volatile noexcept;
void notify_one() noexcept;
void notify_all() volatile noexcept;
void notify_all() noexcept;
notify_token<volatile atomic<T>> get_notify_token() volatile noexcept;
notify_token<atomic<T>> get_notify_token() noexcept;
};
}

```

Modify `[atomics.types.operations]` as follows:

Add the following at the end of the section:

```

notify_token<volatile atomic<T>> get_notify_token() volatile noexcept;
notify_token<atomic<T>> get_notify_token() noexcept;

```

Effects: None

Returns: An atomic notify token that can be used to unblock the execution of atomic waiting operations on `*this`.

Modify `[atomics.types.int]` as follows:

```

namespace std {
    template<> struct atomic<integral> {
        ...

        void wait(integral, memory_order = memory_order::seq_cst) const noexcept;
        void notify_one() volatile noexcept;
        void notify_one() noexcept;
        void notify_all() volatile noexcept;
        void notify_all() noexcept;
        notify_token<volatile atomic<integral>> get_notify_token() volatile noexcept;
        notify_token<atomic<integral>> get_notify_token() noexcept;
    };
}

```

Modify `[atomics.types.float]` as follows:

```

namespace std {
    template<> struct atomic<floating-point> {

```

```

...

void wait(floating-point, memory_order = memory_order::seq_cst) const noexcept;
void notify_one() volatile noexcept;
void notify_one() noexcept;
void notify_all() volatile noexcept;
void notify_all() noexcept;
notify_token<volatile atomic<floating-point>> get_notify_token() volatile
notify_token<atomic<floating-point>> get_notify_token() noexcept;
};
}

```

Modify `[atomics.types.pointer]` as follows:

```

namespace std {
    template<class T> struct atomic<T*> {
        ...

        void wait(T*, memory_order = memory_order::seq_cst) const noexcept;
        void notify_one() volatile noexcept;
        void notify_one() noexcept;
        void notify_all() volatile noexcept;
        void notify_all() noexcept;
        notify_token<volatile atomic<T*>> get_notify_token() volatile noexcept;
        notify_token<atomic<T*>> get_notify_token() noexcept;
    };
}

```

Modify `[util.smartptr.atomic.shared]` as follows

Modify the synopsis as follows:

```

namespace std {
    template<class T> struct atomic<shared_ptr<T>> {
        ...

        void wait(shared_ptr<T> old, memory_order order = memory_order::seq_cst)
        void notify_one() noexcept;
        void notify_all() noexcept;
        notify_token<atomic<shared_ptr<T>>> get_notify_token() noexcept;
    };
}

```

```
};  
}
```

Add the following to the end of the section:

```
notify_token<atomic<shared_ptr<T>>> get_notify_token() noexcept;
```

Effects: None.

Returns: An atomic notify token that can be used to unblock the execution of atomic waiting operations on `*this`.

Modify `[util.smartptr.atomic.weak]` as follows

Modify the synopsis as follows:

```
namespace std {  
    template<class T> struct atomic<weak_ptr<T>> {  
        ...  
  
        void wait(weak_ptr<T> old, memory_order order = memory_order::seq_cst) co  
        void notify_one() noexcept;  
        void notify_all() noexcept;  
        notify_token<atomic<weak_ptr<T>>> get_notify_token() noexcept;  
    };  
}
```

Add the following to the end of the section:

```
notify_token<atomic<weak_ptr<T>>> get_notify_token() noexcept;
```

Effects: None.

Returns: An atomic notify token that can be used to unblock the execution of atomic waiting operations on `*this`.

Modify `[atomics.flag]` as follows

Modify the synopsis as follows:

```
namespace std {  
    struct atomic_flag {
```

```

...

void wait(bool, memory_order order = memory_order::seq_cst) const noexcept;
void notify_one() noexcept volatile;
void notify_one() noexcept;
void notify_all() noexcept volatile;
void notify_all() noexcept;
notify_token<volatile atomic_flag> get_notify_token() noexcept volatile;
notify_token<atomic_flag> get_notify_token() noexcept;
};
}

```

Add the following to the end of the section:

```

notify_token<volatile atomic_flag> get_notify_token() volatile noexcept;
notify_token<atomic_flag> get_notify_token() noexcept;

```

Effects: None.

Returns: An atomic notify token that can be used to unblock the execution of atomic waiting operations on `*this`.

Add new section under `[atomics]`

Atomic notify tokens

`[atomics.notifytoken]`

General `[atomics.notifytoken.general]`

An *atomic notify token* is a token obtained from an atomic object, `M`, that provides the ability for the caller to later perform an atomic notifying operation on the atomic object, `M`, without regard to whether `M`'s lifetime has ended.

An atomic notifying operation on an atomic notify token obtained from an atomic object, `M`, has no effect if `M`'s lifetime has ended as all atomic waiting operations on `M` are required to happens-before `M`'s lifetime ends.

All copies of an atomic notify token obtained from atomic object `M` can be used to unblock atomic waiting operations on `M`.

Class template `notify_token`

`[atomics.notifytoken.generic]`

```
namespace std {
    template<class Atomic> struct notify_token {
    public:
        notify_token(const notify_token&) noexcept = default;
        notify_token& operator=(const notify_token&) noexcept = default;

        void notify_one() const noexcept;
        void notify_all() const noexcept;
    };
}
```

```
void notify_one() const noexcept;
```

Effects: If `*this` was obtained from an atomic object, `M`, then unblocks the execution of at least one atomic waiting operation on `M` that is eligible to be unblocked (`[atomics.wait]`) by this call, if any such atomic waiting operations exist. Otherwise has no effect. [[Note: If the object `M`'s lifetime has ended then there will not be any atomic waiting operations that are eligible to be unblocked and so this call will have no effect.]]

Remarks: This function is an atomic notifying operation (`[atomics.wait]`).

```
void notify_all() const noexcept;
```

Effects: If `*this` was obtained from an atomic object, `M` then unblocks the execution of all atomic waiting operations on `M` that are eligible to be unblocked (`[atomics.wait]`) by this call. Otherwise has no effect. [[Note: If the object `M`'s lifetime has ended then there will not be any atomic waiting operations that are eligible to be unblocked and so this call will have no effect.]]

Remarks: This function is an atomic notifying operation (`[atomics.wait]`).

Wording changes to apply to P2689R2

If both this paper and the paper P2689R2 are merged into the working draft then apply the following changes to the wording added by P2689R2.

Modify `[atomics.ref.bound.general]` as follows

Modify synopsis to add `get_notify_token()` method:

```
template <class T, memory_order MemoryOrder>
struct atomic-ref-bound { // exposition only

    // [other members omitted for brevity]

    void wait(T old) const noexcept;
    void notify_one() const noexcept;
    void notify_all() const noexcept;
    notify_token<atomic-ref-bound> get_notify_token() noexcept;
};
```

Modify `[atomics.ref.bound.ops]` as follows

Append the following to the end of this section:

```
notify_token<atomic-ref-bound> get_notify_token() noexcept;
```

Effects: Equivalent to `ref.get_notify_token()`.

References

SG1 mailing list thread on the issue <https://lists.isocpp.org/parallel/2020/07/3270.php>

P1726R5 - "Pointer lifetime-end zap and provenance, too" (Paul McKenney, Maged Michael, et. al.) <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1726r5.pdf>

P2188R1 - "Zap the Zap: Pointers are sometimes just bags of bits" <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2188r1.html>